Studio Nx

Backend(FastAPI/Python) 29 — authenticate() · invalid_credentials()

2026-10-12

Backend(FastAPI/Python) 29 — `authenticate()` • `invalid_credentials()`

항목	내용
파일	`studionx/backend/app/services/auth_service.py`
작업	P1-05 로그인: 이메일과 비밀번호 확인 (PRD 01 FR-103, FR-108)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	28 `decode_refresh_token()` · 30 `get_active_user()`

1. 하는 일

로그인 모달에 입력한 이메일과 비밀번호가 맞는지 확인합니다. 맞으면 회원을 돌려주고 **마지막 로그인 시각**을 기록합니다.

경우	결과	로그인 모달 문구
이메일과 비밀번호가 맞고, 활동 회원임	`User`를 돌려주고 `last_login_at`을 지금으로 기록합니다	(로그인됩니다)
비밀번호가 틀림	401 `INVALID_CREDENTIALS`	Email or password is incorrect.
없는 이메일	401 `INVALID_CREDENTIALS`이며, 위와 **문구도 같고 걸리는 시간도 같습니다**	Email or password is incorrect.
탈퇴한 회원 (비밀번호가 맞아도)	401 `INVALID_CREDENTIALS`이며, 위와 같습니다	Email or password is incorrect.
정지된 회원이고 비밀번호가 **맞음**	403 `ACCOUNT_DISABLED`	This account is disabled.
정지된 회원이고 비밀번호가 틀림	401 `INVALID_CREDENTIALS`	Email or password is incorrect.

함수	역할
`authenticate()`	확인하고 기록합니다
`invalid_credentials()`	401 오류를 만듭니다 (네 곳에서 같은 오류를 씁니다)

2. 언제 쓰나

쓰는 곳	화면
P1-05 `POST /auth/login` (32)	home.html 로그인 모달의 **Sign In** 버튼

32번이 이 함수로 회원을 확인한 뒤, 26번과 27번으로 토큰 두 개를 만들어 줍니다.

3. 코드

import (파일 맨 위)

```
from datetime import UTC, datetime, timedelta
```

```
from sqlalchemy import select
from sqlalchemy.exc import IntegrityError
from sqlalchemy.orm import Session

from app.core.errors import AppError, ErrorCode
from app.core.security import check_password_rules, hash_password, verify_password
from app.models import User
from app.models.user import UserStatus
from app.schemas.auth import PRIVACY_VERSION, AvailabilityResponse, SignupRequest
```

`auth_service.py` 맨 위의 import입니다. **`verify_password`를 이번에 추가했습니다.** 이 함수는 `datetime`, `UTC`, `select`, `Session`, `AppError`, `ErrorCode`, `verify_password`, `User`, `UserStatus`를 사용합니다.

함수

```
# 없는 이메일로 로그인할 때 비교에 쓰는 가짜 해시 (bcrypt 비용 12 — 진짜 해시와 같은 시간이 걸림).
# 어떤 비밀번호와도 맞지 않는다. 비밀값이 아니므로 코드에 두어도 됨.
DUMMY_PASSWORD_HASH = "$2b$12$1YCmgxF3pcmPkm7MWpsgVuOD.w6LlO7o43SuOZLbLZkz.glHDfmva"


def authenticate(db: Session, email: str, password: str) -> User:
    user = db.scalar(select(User).where(User.email == normalize_email(email)))
    if user is None:
        verify_password(password, DUMMY_PASSWORD_HASH)  # 없는 이메일도 같은 시간(약 0.2초)이 걸리게
        raise invalid_credentials()
    if not verify_password(password, user.password_hash):
        raise invalid_credentials()
    if user.status == UserStatus.DELETED:
        raise invalid_credentials()
    if user.status == UserStatus.SUSPENDED:
        raise AppError(ErrorCode.ACCOUNT_DISABLED, 403, "This account is disabled.")
    user.last_login_at = datetime.now(UTC).replace(tzinfo=None)
    db.commit()
    return user


def invalid_credentials() -> AppError:
    return AppError(ErrorCode.INVALID_CREDENTIALS, 401, "Email or password is incorrect.")
```

문법 풀이

import 문법	뜻
`from app.core.security import …, verify_password`	16번의 비교 함수를 가져옵니다

식	뜻
`verify_password(password, DUMMY_PASSWORD_HASH)`	결과를 **쓰지 않고** 부르기만 합니다. 시간만 쓰게 하려는 것입니다
`raise invalid_credentials()`	함수가 만든 오류를 던집니다 (21번의 `unavailable_error`와 같은 방식)
`user.last_login_at = …` → `db.commit()`	회원 행의 마지막 로그인 시각을 바꿔 저장합니다
`.replace(tzinfo=None)`	DB 칸(`DATETIME`)에는 시간대 표시가 없으므로, UTC 시각에서 표시를 뗍니다 (19번과 같음)

4. 흐름

1. 이메일을 17번으로 정리한 뒤 회원을 찾습니다.
2. 회원이 없으면 가짜 해시로 한 번 비교하고(시간 맞추기) 401로 응답합니다.
3. 비밀번호가 틀리면 401로 응답합니다.
4. 탈퇴한 회원이면 401로 응답합니다 (같은 문구).
5. 정지된 회원이면 403으로 응답합니다.
6. 마지막 로그인 시각을 기록하고 회원을 돌려줍니다.

5. 왜 이렇게

선택	이유
틀린 이유를 알려 주지 않음	"이메일이 없음"과 "비밀번호가 틀림"을 구분해서 알려 주면, 남의 이메일이 가입되어 있는지 알아낼 수 있습니다 (FR-103 통일 문구)
없는 이메일도 가짜 해시로 비교	비교(약 0.2초)를 건너뛰면 없는 이메일에만 빨리 응답하게 됩니다. 그러면 응답 **시간**으로 가입 여부가 드러납니다. 이것을 막습니다 (16 §5에 적어 둔 일)
가짜 해시를 코드에 상수로 둠	서버가 켜질 때마다 0.2초씩 들여서 만들 필요가 없습니다. 비밀번호가 아니라 "아무것과도 맞지 않는 해시"라서 드러나도 문제가 없습니다. 비용 값이 12라서 진짜 해시와 걸리는 시간이 같습니다
탈퇴한 회원에게도 같은 401	탈퇴한 계정이 있었다는 사실조차 알려 주지 않습니다 (18번의 추적 방지와 같은 생각)
정지(403)는 **비밀번호가 맞은 뒤에만** 알림 (템플릿 순서 그대로)	틀린 비밀번호로는 "이 이메일은 정지되었다"는 사실을 알 수 없습니다. 비밀번호를 아는 본인에게만 정지 사실을 알려 줍니다
실패하면 `last_login_at`을 바꾸지 않음	성공한 로그인만 기록합니다

참고 (탈퇴 3일 뒤): 18번으로 비켜 둔 행은 이메일이 `deleted+…`로 바뀌어서 원래 이메일로는 찾을 수조차 없습니다. 또 P1-09 정리 작업이 비밀번호 해시를 `"!"`로 바꾸므로, 16번은 항상 `False`를 돌려줍니다.

6. 시험 (`tests/test_auth_service.py`에 7개를 추가해서 모두 44개, 실제 MySQL로 실행)

시험	확인한 것
성공	그 회원을 돌려주고, `last_login_at`이 지금(5초 안)으로 기록됩니다
대소문자와 공백	`" LOGIN@Example.com "`으로도 로그인됩니다 (17)
비밀번호가 틀림	401 "Email or password is incorrect."이고, `last_login_at`은 그대로입니다
없는 이메일	같은 401과 같은 문구이고, 걸린 시간이 0.1초 이상입니다 (가짜 해시 비교를 했다는 뜻입니다)
탈퇴한 회원, 맞는 비밀번호	401입니다 (틀린 비밀번호와 구분되지 않습니다)
정지된 회원	맞는 비밀번호면 403 "This account is disabled.", 틀린 비밀번호면 401입니다
가짜 해시	`""`, `"password"`, `"Abcdef1!"` 등 어떤 값과도 맞지 않습니다

진짜 해시로 로그인을 시험할 수 있도록, 시험 도우미 `add_user()`가 `password_hash`를 바꿔 넣을 수 있게 고쳤습니다.

전체 시험 209개가 통과했고, `ruff` 검사도 통과했습니다. 시험이 끝난 뒤 남은 행은 0개입니다.